

IMPLEMENTACIÓN OLAP – DISEÑO GENERAL

ARQUITECTURA DE ANÁLISIS OLAP CON SPARK / HIVE Y METABASE

VERSION 1.2

King Bustamante Jenny

Ulate Obando Evelyn Vanessa

Contents

1. Introducción (Contexto)	3
2. Modelo General	3
capa OLTP.....	3
capa analítica	3
3. Arquitectura General.....	4
4. Decisiones de Diseño.....	5
5. Capas de Datos.....	7
5.1. Interacción entre servicios.....	8
6. Visualización (Metabase y Folium)	9
Metabase (v0.50.21)	9
Folium	11
Limitaciones y escalabilidad	12

1. INTRODUCCIÓN (CONTEXTO)

El presente documento describe la implementación del Proyecto 2 de Bases de Datos II: *Análisis y Procesamiento OLAP*, construido como extensión del sistema de gestión de restaurantes desarrollado en el Proyecto 1.

El sistema base (P1) es una API REST en Go que gestiona usuarios, restaurantes, pedidos y reservaciones sobre PostgreSQL, con soporte para MongoDB, Redis y Elasticsearch, expuesta mediante NGINX y protegida por Keycloak. Dicho sistema constituye la fuente de datos operacionales (OLTP) sobre la cual se construye la capa analítica del presente proyecto.

El Proyecto 2 agrega:

- Data Warehouse dimensional en Apache Hive con esquema estrella.
- Pipeline de análisis en Apache Spark (tendencias, horarios pico, crecimiento mensual).
- Orquestación automática con Apache Airflow (@daily).
- Grafo de co-compra en Neo4j.
- Módulo de enrutamiento con asignación de repartidores.
- Visualización consolidada en Metabase, conectado tanto a PostgreSQL como al DW real vía Spark Thrift Server.

2. MODELO GENERAL

El sistema maneja dos capas de datos funcionalmente distintas pero derivadas de la misma fuente cruda:

- la capa operativa (OLTP) heredada del Proyecto 1,
- la capa analítica construida en el Proyecto 2.

La separación evita que las cargas de lectura analítica interfieran con las transacciones operativas, y que permite evolucionar el modelo analítico sin tocar el esquema relacional que el API de Go ya usa en producción. Un efecto secundario de esta decisión es que la capa analítica funciona únicamente para Postgres, pero mantiene la posibilidad de poder implementar con mongo a futuro (creando los respectivos traductores).

CAPA OLTP

Vive en PostgreSQL (public.*). Contiene las entidades del dominio:

- usuarios, restaurantes, órdenes, ítems de orden y reservaciones.

Este esquema es propiedad del sistema Go (GORM). Cualquier tabla que no esté declarada como modelo Go en el repositorio es invisible para el ORM y no recibe CRUD.

CAPA ANALÍTICA

Se divide a su vez en dos pipelines paralelos que nacen del mismo crudo y nunca se leen entre sí:

El primer pipeline es el **Data Warehouse en Hive (restaurant_dw)**. El ETL lee de vía JDBC y aplica transformaciones de desnormalización, calcula sesiones de co-compra y escribe un esquema

estrella en formato Parquet. Esta administra el catálogo de tablas y vistas, gestionado por un metastore Derby embebido.

Seguidamente crea 5 vistas OLAP precomputadas

- I. v_ingresos_mes_categoria
- II. v_estadisticas_pedidos_estado
- III. v_actividad_clientes_zona

y dos más son el producto final de este pipeline.

El segundo pipeline es el **esquema analítico en Postgres (analytics.*)**. La cual es un paso intermedio- crudo de como se ejecutan los jobs de Spark solos, computan tendencias de consumo, horarios pico y crecimiento mensual, y escriben los resultados como tablas relacionales en Postgres. Este pipeline solo funciona para el módulo de enrutamiento también y existe para evidenciar el procesamiento Spark y para dar a Metabase acceso directo a coordenadas geográficas que Hive no contiene.

3. ARQUITECTURA GENERAL

El sistema se despliega como un conjunto de contenedores Docker organizados en cuatro perfiles de ejecución:

Perfil	Servicios incluidos
app	api, nginx, keycloak, redis, elasticsearch, search-service
analytics	spark, metabase, airflow, neo4j
dw	spark-thrift
mongo	cluster MongoDB (3 shards)

Los perfiles permiten levantar únicamente la capa necesaria según el caso de uso, sin cargar el stack completo. El orquestador Airflow reside en analytics porque requiere el socket Docker para lanzar el contenedor Spark.

Inventario de servicios:

Servicio	Imagen	Puerto	Rol
db	postgres:18-alpine	5432	OLTP principal
api	build local (Go)	—	API REST · 2 réplicas
nginx	nginx:alpine	8000	API Gateway
keycloak	keycloak:23.0.7	8082	Autenticación
redis	redis:alpine	6379	Caché

elasticsearch	elasticsearch:8.12.0	9200	Motor de búsqueda
spark	pyspark-notebook:spark-3.5.0	8888	ETL + análisis
spark-thrift	pyspark-notebook:spark-3.5.0	10000	Sirve DW por JDBC
metabase	metabase:v0.50.21	3000	Dashboards
airflow	airflow:2.8.1	8081	Orquestador
neo4j	neo4j:5.15.0	7474 / 7687	Grafo

A razgos generales la implementación de del proceso analítico se puede entender por medio del siguiente diagrama lógico:

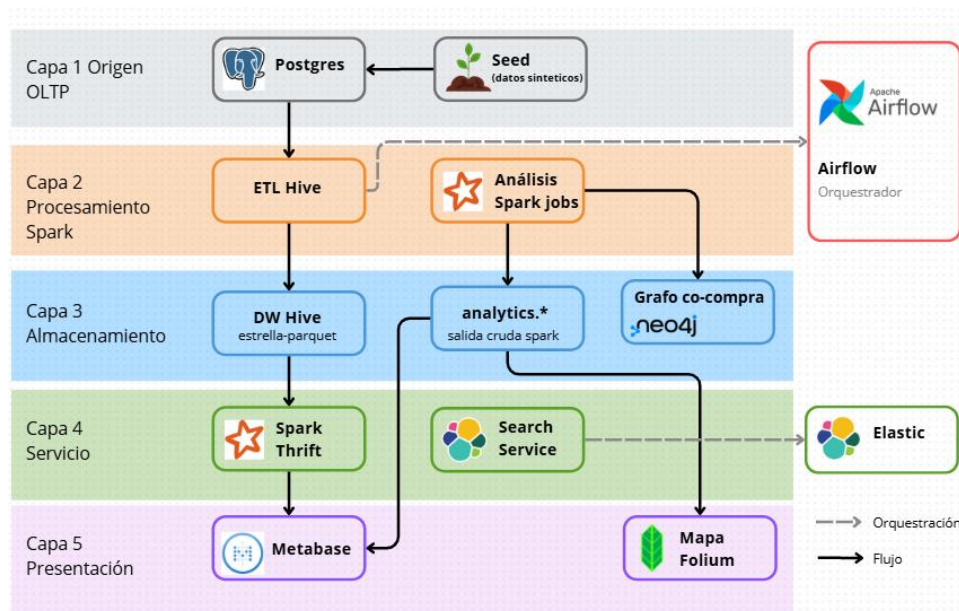


Figura 1: Diagrama Lógico

4. DECISIONES DE DISEÑO

- Metabase conectado al DW vía Spark Thrift Server

Se desplegó un Spark Thrift Server que expone Hive directamente por medio de un JDBC en el puerto 10000. Dado que Metabase v0.50.21 incluye el driver Spark SQL de forma nativa, se elimina la necesidad de plugins externos. Esta decisión mantiene la coherencia del pipeline: los datos analíticos del DW existen únicamente en Hive.

- Dos pipelines paralelos sobre el mismo origen

Hive y analytics.* son pipelines paralelos e independientes, ambos con origen en public.* de PostgreSQL. Hive es el DW dimensional (consultado vía Thrift). analytics.* contiene las salidas

directas de los jobs Spark y del módulo de rutas (consultado directamente por Metabase en PostgreSQL). Hive nunca lee de analytics, ni viceversa.

- **Metastore Derby: restricción de escritura única**

Apache Hive embebido usa Derby como metastore. Derby admite un solo escritor simultáneo, por lo que el ETL (spark_p2) y el Spark Thrift Server (spark_thrift_p2) no pueden correr a la vez. La separación en perfiles analytics vs dw es consecuencia directa de esta limitación.

- **Entidades del seed fuera del modelo Go**

El sistema Go no modela repartidores, recomendaciones de productos ni coordenadas geográficas de restaurantes. Dichas entidades no existen en ningún struct con etiqueta gorm. Tal que el seed las inserta directamente en Postgres sin pasar por el API:

Esta inserta ~55 000 órdenes distribuidas en 8 meses con sesgos realistas (picos nocturnos, crecimiento mensual sostenido), y agrega registros de repartidores con latitud/longitud y recomendaciones producto-a-producto. La tabla recomendaciones se conecta al resto del esquema por FK a products.id; la tabla repartidores se relaciona solo geográficamente (sin FK) con órdenes y restaurantes a través de los cálculos de proximidad en el módulo de enrutamiento. Estas entidades son invisibles al ORM: no hay migración, no hay endpoint, no rompen el coverage del P1.

5. CAPAS DE DATOS

El siguiente diagrama de secuencia ilustra el flujo de datos y requests entre servicios

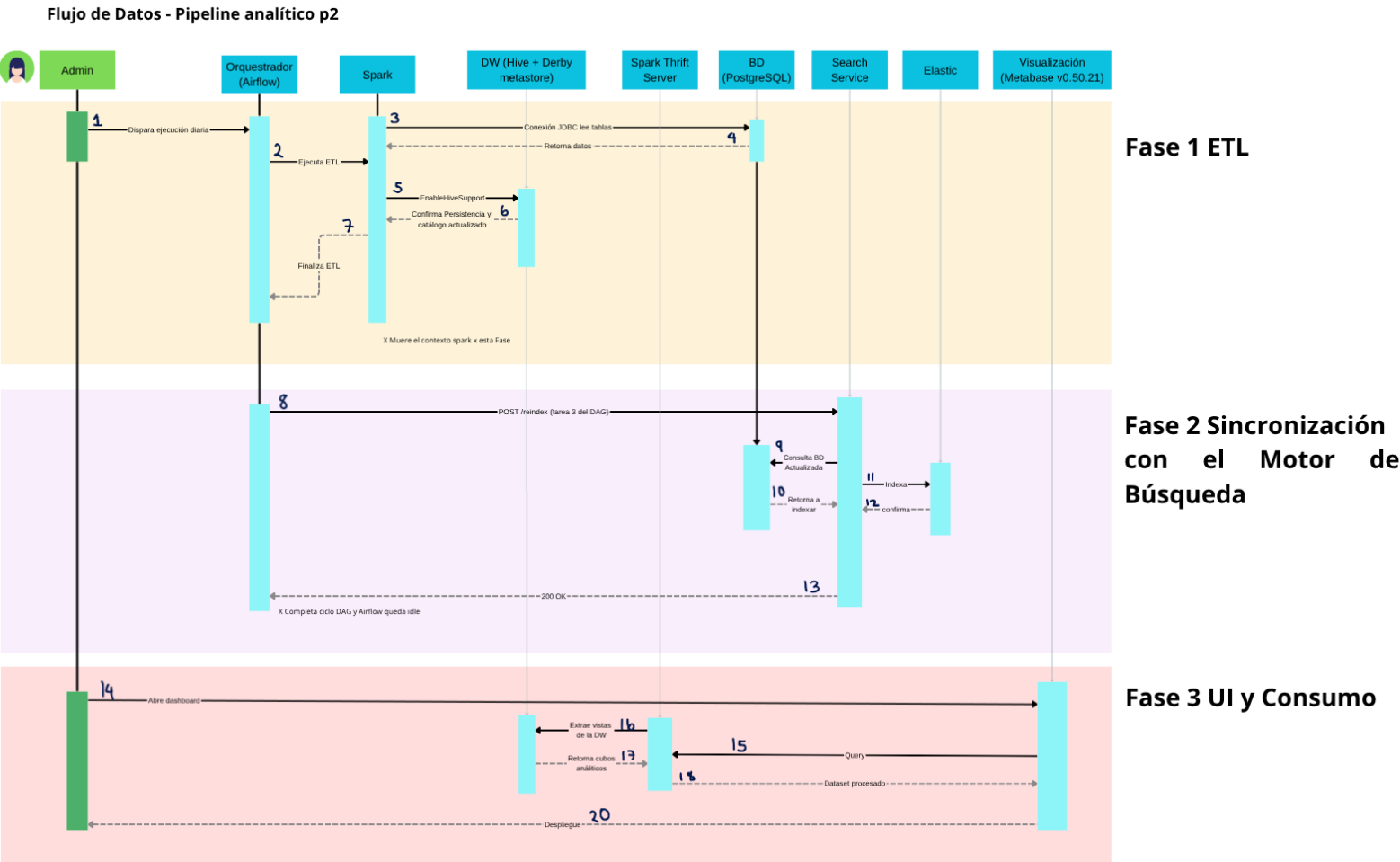


Figura 2: Pipeline Analítico

5.1. INTERACCIÓN ENTRE SERVICIOS

Fase	Flujo
1 ETL diario	Admin dispara el DAG en Airflow, que via Docker ejecuta etl_hive.py en el contenedor Spark. Spark abre conexión JDBC a PostgreSQL, extrae las tablas del esquema public.*, aplica las transformaciones del esquema estrella con enableHiveSupport() y persiste el resultado como Parquet en el DW (Hive). Derby registra el catálogo actualizado. El contexto Spark muere al finalizar el script.
2 Sincronización Elastic	Airflow lanza la tarea de reindex: hace POST /reindex al search-service, que consulta PostgreSQL para obtener los datos actualizados, los empuja a Elasticsearch para reindexar, recibe confirmación, y retorna 200 OK al DAG. El ciclo diario se completa y Airflow queda idle.
3 UI y Consumo	El Admin abre Metabase. Metabase envía queries al Spark Thrift Server vía JDBC :10000; el Thrift extrae las vistas precomputadas del DW en Hive y devuelve los cubos analíticos procesados. Metabase renderiza los dashboards y despliega los reportes al Admin.

El siguiente diagrama ayuda a comprender la topología lógica de conectividad entre los servicios que componen el sistema, mostrando de forma estática quién se comunica con quién y a través de qué protocolo, sin entrar en la dimensión temporal ni en el orden de ejecución.

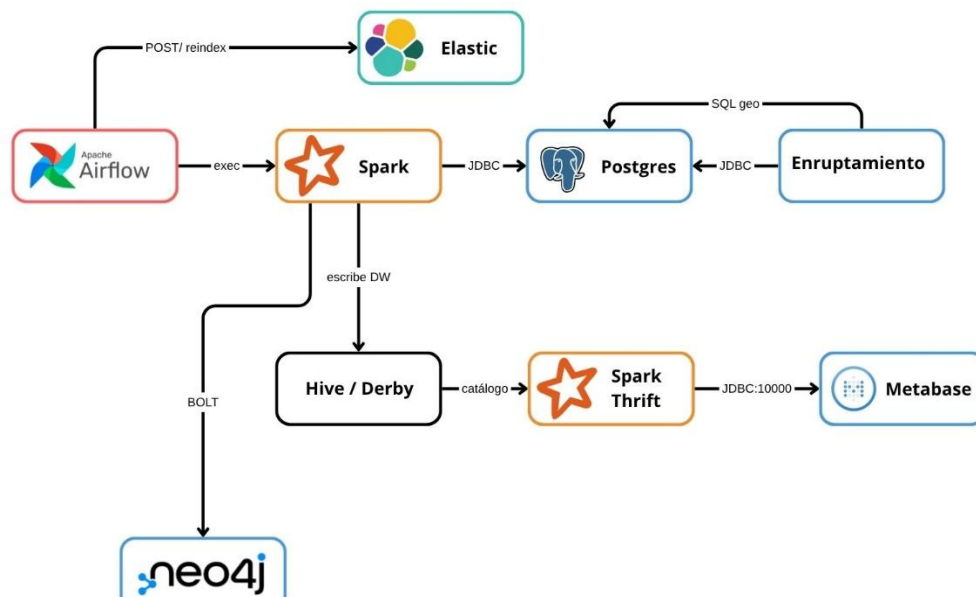


Figura 3: Conectividad individual entre servicios

Aclaraciones del diagrama:

exec — Airflow le manda al Docker que ejecute un comando dentro del contenedor Spark (`docker exec spark_p2 python jobs/...`). Es una llamada al API de Docker.

JDBC — Java Database Connectivity. Es el protocolo estándar que usa Spark para hablar con PostgreSQL. Spark abre un socket TCP al puerto 5432, envía queries SQL y recibe los resultados en formato tabular. El driver que lo hace posible es el `jar postgresql-42.7.3`.

write DW — Operación interna de Spark: al tener `enableHiveSupport()` activo, Spark escribe los DataFrames directamente como archivos Parquet en `spark-warehouse/` y registra los metadatos de las tablas en Derby. Todo ocurre dentro del mismo proceso.

catálogo — Es el **metastore Derby embebido**: un archivo en disco que contiene el catálogo de tablas y vistas de Hive (nombres, esquemas, rutas a los parquet). Tanto el ETL como el Thrift Server leen y escriben ese mismo archivo, lo que los hace dependientes entre sí.

JDBC :10000 — Igual que JDBC a Postgres, pero aquí el destino es el Spark Thrift Server que habla el protocolo **HiveServer2** sobre ese puerto. Metabase usa el driver `Spark SQL built-in` de v0.50.21 para conectarse.

SQL geo — Una query SQL normal de Metabase directo a Postgres, separada del DW, para obtener las coordenadas latitud/longitud que Hive no almacena.

Bolt — Protocolo binario propietario de Neo4j (puerto 7687). Es lo que usa el driver Python `neo4j` para crear nodos y relaciones en el grafo. Más eficiente que HTTP para operaciones de grafo masivas.

POST /reindex — Llamada HTTP REST que Airflow hace al `search-service`, que a su vez actualiza los índices de Elasticsearch.

6. VISUALIZACION (METABASE Y FOLIUM)

La capa de visualización expone los resultados del pipeline analítico a través de dos herramientas con propósitos distintos: Metabase para el análisis multidimensional sobre el Data Warehouse, y Folium para la representación geoespacial de las rutas de entrega.

METABASE (V0.50.21)

Se configura una conexión Spark Thrift Server vía JDBC en el puerto 10000, que expone las vistas OLAP precomputadas del DW en Hive. A través de ella se construyen dashboards que cubren ingresos por categoría y mes (`v_ingresos_mes_categoria`), estadísticas de pedidos por estado (`v_estadisticas_pedidos_estado`), actividad de clientes por zona geográfica (`v_actividad_clientes_zona`), tendencias de consumo y crecimiento mensual.



Figura 3: Evidencia del Dashboard en Metabase



Figura 4: Evidencia 2 del Dashboard en Metabase

FOLIUM

Genera un mapa interactivo en HTML a partir de las asignaciones calculadas por el módulo de enrutamiento. Para cada repartidor se traza la secuencia de entregas obtenida con el algoritmo de vecino más cercano usando distancia haversine sobre las coordenadas reales de los restaurantes y usuarios del seed. El resultado es un archivo HTML autocontenido que muestra las rutas asignadas por repartidor, exportable sin dependencia de servicios externos.

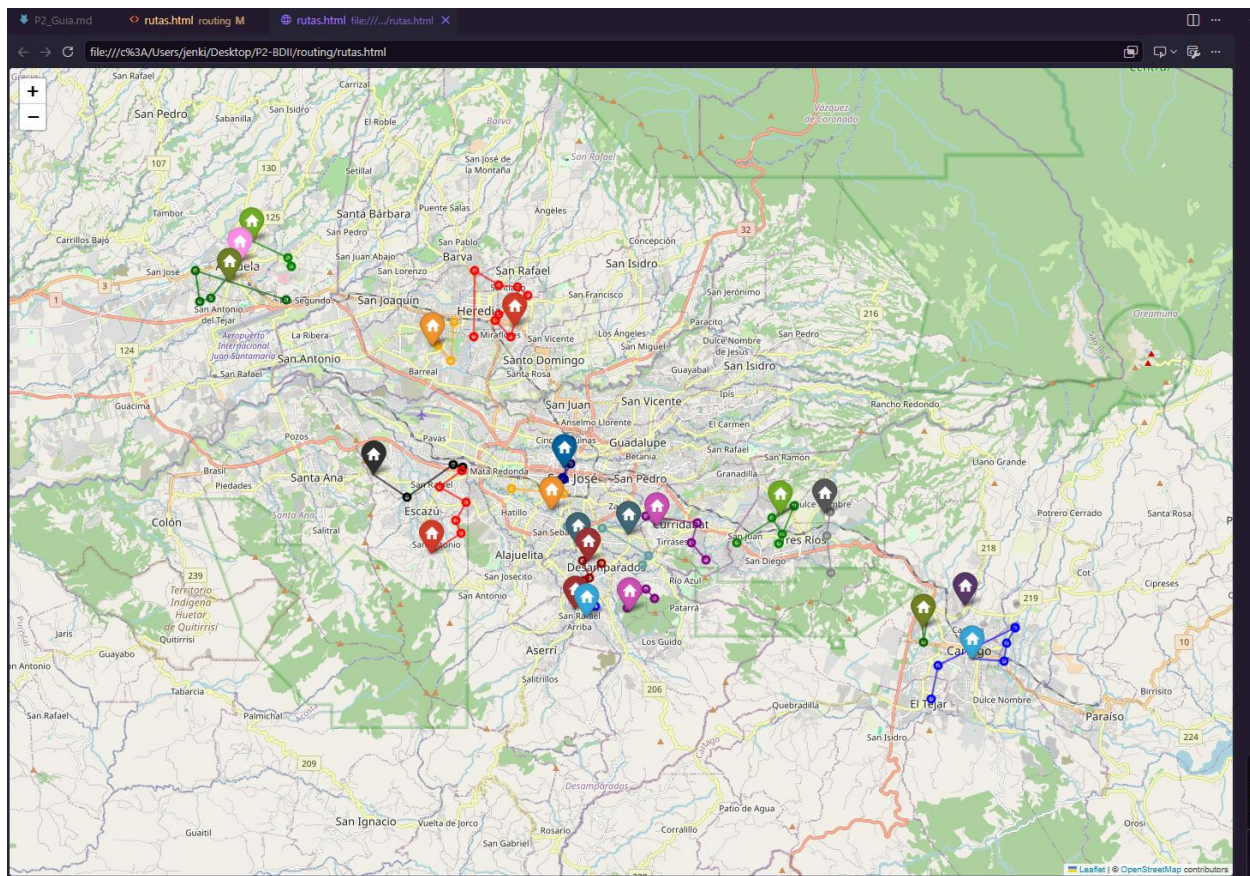


Figura 5: Evidencia Mapa Folium

7. LIMITACIONES Y ESCALABILIDAD

El principal cuello de botella de esta implementación es el metastore Derby. Derby es una base de datos embebida de un solo escritor, lo que significa que el ETL y el Spark Thrift Server no pueden operar simultáneamente: si el Thrift está activo sirviendo consultas a Metabase cuando Airflow dispara el pipeline, el job de Spark falla al intentar escribir el catálogo. En el contexto de desarrollo esto se gestiona manualmente deteniendo el Thrift antes de correr el ETL, pero es una limitación que no escala.

En un ambiente productivo esta restricción se elimina reemplazando Derby por un metastore externo sobre PostgreSQL o MySQL. Un metastore relacional acepta lecturas y escrituras concurrentes, lo que permite que el ETL actualice el Data Warehouse mientras el Thrift Server sigue sirviendo las particiones anteriores sin interrupción. Al finalizar el ETL, el Thrift detecta automáticamente las tablas y particiones nuevas sin necesidad de reinicio.

Con un metastore externo también se habilita la posibilidad de escalar horizontalmente el Thrift Server: múltiples réplicas del servicio apuntan al mismo warehouse en disco y al mismo metastore compartido, y un balanceador de carga distribuye las consultas entrantes de Metabase entre ellas. Este es el modelo que implementan plataformas como Databricks con su Hive Metastore administrado, donde decenas de clientes consultan el mismo Data Warehouse en paralelo sin degradación de rendimiento.